

SepsisCare 期末汇报详细中文演讲稿

ICU 脓毒症风险分层、动态表型与多端软件交付

汇报日期：2026 年 6 月 10 日 对齐正式 17 页 PPT

版本说明：详细口语版，可直接照稿讲；保留证据边界，避免把课程演示说成临床验证。

整体讲法：这份稿子按 12 到 15 分钟准备，语速正常即可。它故意写得比短稿更细，因为真正上台时，人容易漏掉“为什么”和“所以”。如果现场只给 8 到 10 分钟，看每页的“这一页我想让老师记住”和最后的压缩版即可。

说人话的原则：不要一上来堆 S0、S7、GroupDRO。先说“我解决什么问题”，再说“为什么要这么做”，最后才说“我做到了什么”。老师听到模型名词时，最好已经知道它在系统里承担什么作用。

目录

1 开场：先把项目说清楚	2
第 1 页：SepsisCare 总览	2
第 2 页：问题与边界	2
2 数据与研究：从时间序列到动态表型	2
第 3 页：数据基础	2
第 4 页：S0 到 S7 研究路线	3
第 5 页：S0 到 S1.5：表示基础	3
第 6 页：S2 到 S5：从表型到实时使用	4
第 7 页：S6 到 S7：最终鲁棒训练	4
第 8 页：动态表型	4
第 9 页：最终 S7 模型	5
第 10 页：关键结果与证据边界	5
3 软件系统：从模型到可用产品	6
第 11 页：多角色产品体验	6
第 12 页：软件架构与 API 流	6
第 13 页：网络安全加固	6
第 14 页：数据污染与 Artifact 完整性	7
第 15 页：部署编排	7
第 16 页：最终交付物	7
4 收束：主动讲限制，反而更可信	8
第 17 页：限制与下一步	8
5 现场压缩与临场提醒	8

1 开场：先把项目说清楚

第 1 页：SepsisCare 总览

这一页我想让老师记住：SepsisCare 是一个把 ICU 时间序列研究做成可安装软件、可部署模型服务、可验证交付包的课程项目。

上台提示：开场不要太快。先让老师知道你做的不是单个模型，也不是单个 App，而是一条完整链路。

可以这样讲：各位老师好，我今天汇报的项目是 SepsisCare。这个项目的主题是 ICU 脓毒症风险分层、动态表型分析，以及多平台软件交付。

我先用一句比较直白的话概括：SepsisCare 不是只训练了一个模型，也不是只写了一个界面。它做的是一条完整链路：前面从 ICU 时间序列里学习患者状态，中间把状态变成风险、表型和相似病例，后面再把这些能力包装成医生端、研究端、家属端、管理员端和训练终端可以使用的软件系统。

这也是为什么第一页我放了三个关键词：research model、working app、security evidence。研究模型负责回答“患者现在像什么状态”；working app 负责回答“这个东西怎么给人用”；security evidence 负责回答“交付出去以后，怎么证明它没有明显的数据污染、密钥泄漏和部署假成功”。

这里三个总览数字可以先记住：S7 模型声明训练患者数是 347,634；最终动态表型 Macro F1 是 0.956；死亡风险 AUROC 是 0.848。这些数字说明模型在课程演示和工程验证里是可用的。但我也会一直强调，它们不是临床投产证明，不是医疗器械认证，也不是独立外部临床验证。

过渡句：所以接下来我会按这个顺序讲：先讲问题和边界，再讲数据和模型路线，然后讲软件系统，最后讲安全、部署和还没完成的边界。

第 2 页：问题与边界

这一页我想让老师记住：这个项目要解决的是“ICU 状态变化太快、静态分数不够用”的问题，但系统输出只能作为课程演示和工程验证。

可以这样讲：SepsisCare 选择脓毒症，是因为脓毒症在 ICU 里非常典型：它变化快，牵涉多个器官系统，而且很多风险不是单个时间点就能看清楚。比如同样是高风险患者，有的人主要表现为循环休克，有的人主要是呼吸衰竭，有的人则是炎症指标和乳酸逐渐恶化。如果只给一个静态分数，医生很难知道这个分数背后的状态变化。

所以我把问题拆成两层。第一层是研究问题：怎么从 ICU 48 小时多变量时间序列里，学到一个稳定的患者表示和窗口表示。这里的“窗口表示”很重要，因为患者不是只有一个最终状态，而是在 48 小时里不断变化。

第二层是工程问题：模型训练出来以后，怎么让它真的能被打开、能被调用、能被验证。一个课程项目如果最后只停在 notebook，演示效果会比较弱。所以我把它做成多端客户端、统一 HTTP API、模型服务、训练终端和部署脚本。

但我也必须把边界讲清楚。SepsisCare 的输出不能作为自主临床医嘱，不能替代医生诊断，也不是受监管医疗器械。所有结果都只用于课程演示、工程验证和方法展示。这个边界不是最后才补一句，而是贯穿了我的报告、应用文案、部署说明和安全审计。

过渡句：有了这个边界以后，我们再来看模型吃的是什么数据，以及为什么要做动态表型。

2 数据与研究：从时间序列到动态表型

第 3 页：数据基础

这一页我想让老师记住：四个 ICU 数据源被统一成同一种 48 小时接口；最终交付包只带脱敏演示数据和模型产物，不带受限原始数据库。

可以这样讲：数据这一页看起来像数字列表，但它其实决定了后面模型为什么不能写得太简单。

我用了四个 ICU 数据来源。PhysioNet 2012 有 11,986 名患者，是主表型来源；PhysioNet 2019 有 40,331 名患者，主要提供 sepsis 相关辅助信息；MIMIC-IV 有 94,458 名患者，是比较重要的外部 ICU 来源；eICU 有 200,859 名患者，它的规模最大，也更能体现多中心差异。最终 S7 训练元数据里声明的总训练患者数是 347,634。

这些来源不是直接拼起来就能用。不同数据库变量名不同，采样频率不同，缺失模式也不同。所以我先把它们都映射成统一的 48 小时 ICU 时间序列接口。模型看到的是连续值、观测掩码、静态特征、干预代理变量、可用标签和来源信息。

这里还有一个实际问题：不是每个数据源都有同样的标签。比如有的来源更适合做 sepsis 辅助任务，有的来源适合做死亡风险，有的来源只有部分干预信息。因此训练时不能硬给所有样本都造标签，而是用 masked loss：哪个任务标签存在，就计算哪个任务的损失。

最后交付时，我没有把受限原始 ICU 数据库打包进去。交付包里是模型权重、读出器、部署服务、报告和脱敏演示 runtime 数据。这样做是为了避免把课程项目做成数据泄漏风险点。

过渡句：数据统一以后，下一步不是马上做分类，而是先建立一条研究路线：一步一步证明这个时间序列表示是有用的。

第 4 页：S0 到 S7 研究路线

这一页我想让老师记住：S0 到 S7 不是炫技编号，而是一条逐步修问题的路线：先能表示，再能解释，再能实时使用，最后能部署。

可以这样讲：这一页是整个研究部分的地图。这里的 S0 到 S7 不是为了显得复杂，而是因为我在做的过程中，每一步都遇到了一个具体问题，然后下一步去修它。

S0 做最基础的数据标准化：把 ICU 数据对齐到 48 小时，做患者级切分，保存训练集统计量。这个阶段最关心的是别泄漏，比如不能用验证集或测试集的信息去做标准化。

S1 做 masked reconstruction。说人话就是：我先把一部分已经观测到的 ICU 数值藏起来，让模型去猜回来。如果模型连这个都学不好，那直接拿它做死亡预测就很危险。S1.5 又加入对比学习，把同一个患者的不同增强视图拉近，让 embedding 更稳定。

S2 到 S5 解决解释和实时问题。S2 把 48 小时切成多个滚动窗口，发现 P0 到 P3 的动态表型。S3 看这个表型结构能不能从一个中心迁移到另一个中心。S3.5 做风险读出和校准。S5 处理实时场景：真实系统在第 10 小时，只能看到前 10 小时的数据，不可能偷看完整 48 小时。

S6 和 S7 是最终模型阶段。S6 引入缺失感知 GRU，因为 ICU 的缺失本身可能有临床意义。S7 再把多来源训练、GroupDRO、表型监督、监督对比学习和辅助任务放在一起，形成最后的可部署模型。

所以这条路线的逻辑是：先让表示站得住，再让表型说得清，再让系统在实时和部署场景里跑得起来。

第 5 页：S0 到 S1.5：表示基础

这一页我想让老师记住：我没有直接做死亡分类，是因为直接分类容易得到一个能出分数、但解释和迁移都很弱的模型。

可以这样讲：这一页讲研究的第一段：表示学习。

S0 是最朴素但很重要的一步。每个 ICU stay 被对齐到入 ICU 后 1 到 48 小时。每个小时都有一组临床变量，比如心率、血压、血氧、乳酸、白细胞、肌酐等；同时也有一个 mask，告诉模型这个值是真的测到了，还是缺失。这里我特别注意患者级切分，因为 ICU 数据里同一个患者可能有多个记录，如果切分不好，会让训练和测试互相污染。

S1 的思路比较像自监督学习。我不急着问模型“这个人会不会死亡”，而是先问它“你能不能理解这条 ICU 时间序列”。具体做法是隐藏一部分观测值，让模型根据上下文重构。这个阶段能帮我判断时间结构是否可学习。

到了 S1.5，我发现只做重构还不够，因为重构学到的 embedding 不一定适合后面的聚类和风险排序。所以我加了对比学习：同一个患者的两种增强视图应该靠近，不同患者则应该区分开。这样表

示空间会更稳定，后面做滚动窗口表型也更有基础。

如果用一句很口语的话说，这一页就是：我先让模型学会“看懂 ICU 轨迹”，再让它去“判断风险”。否则模型可能会给你一个数字，但你很难解释这个数字从哪里来。

第 6 页：S2 到 S5：从表型到实时使用

这一页我想让老师记住：动态表型把模型输出从一个分数，变成一段可以解释的病情轨迹。

可以这样讲：第二段研究是从 embedding 到动态表型。

在 S2 里，我把每个患者的 48 小时轨迹切成 5 个滚动窗口，每个窗口 24 小时，相邻窗口之间有重叠。这样做的好处是，一个患者可以有一串状态，而不是只有一个最终标签。比如早期是 P0 低危稳定型，中间转到 P2 循环休克型，后面又可能转到 P3 呼吸衰竭型。这个序列比单一死亡概率更容易讲清楚。

S3 做的是跨中心验证。简单说，就是不要只在一个中心里聚出几个看起来不错的类，然后就说它有临床意义。我把 Center A 上学到的表型结构拿到 Center B 上看，检查表型比例、死亡率排序、最高风险表型和转移矩阵是否还能保持大致一致。

S3.5 再把 embedding 和统计特征结合起来做死亡风险读出，并做校准。这里的重点不是只看 AUROC，而是看这个风险能不能和表型解释配合起来。

S5 则是为软件部署服务的。真实系统不会等 48 小时结束再告诉你风险，它应该在第 h 小时就用当前能看到的数据判断。所以我加入 horizon augmentation，让训练过程模拟“只看到部分历史”的情况，再加上报警阈值、最小历史长度、连续触发和冷却时间，减少早期误报。

所以这一页要传达的是：表型不是额外装饰，它是模型和临床解释之间的桥。没有这层桥，软件界面就只能显示一个冷冰冰的分数。

第 7 页：S6 到 S7：最终鲁棒训练

这一页我想让老师记住：最终模型把缺失、数据源差异、表型监督和辅助临床任务一起考虑，目的是服务完整系统，而不是只赢一个指标。

可以这样讲：第三段研究是最终模型选择。

ICU 数据有一个很麻烦的特点：缺失不是随机的。比如某些指标测得更频繁，可能说明病情更重；某些指标长期没测，也可能说明临床上认为暂时不需要。所以我在 S6 里用了 missingness-aware GRU。它不只输入数值，还输入 mask、干预代理变量和距离上次观测的时间差。这样模型可以把“没有测”也当成一种信息，而不是简单填 0 或均值。

跨数据源训练也不容易。PhysioNet、MIMIC、eICU 的病人结构、采样习惯和标签质量都不同。如果只是把数据混在一起，大数据源可能会主导训练；如果强行做域不变，又可能把有用的临床差异抹掉。所以我尝试了 source balance、DANN/CORAL、GroupDRO、GRU-D 风格输入和 ensemble。最后更稳定的方案是源均衡采样加 GroupDRO，再配合表型 CE 和监督对比学习。

S7 的目标不是只预测死亡。它同时服务死亡风险、表型一致性、机械通气、剩余 ICU 住院时长和转移矩阵。这样模型输出才能支撑医生端、研究端、训练终端和部署 smoke test。

最终结果里，S7 有 347,634 声明训练患者，模型参数量 31,319，encoder Macro F1 0.971，平滑后的 Macro F1 0.956，下一时段机械通气 AUROC 0.968。这些数字说明它作为课程最终模型是合适的。只是我不会把它包装成“已经临床验证”，因为证据边界不支持这样说。

第 8 页：动态表型

这一页我想让老师记住：动态表型让老师能直观看到：SepsisCare 不是只报高低风险，而是在描述患者状态怎么变。

可以这样讲：这一页可以稍微放慢一点，因为它是模型解释的核心。

我们可以把一个 ICU 患者想象成一条 48 小时的曲线。传统做法可能是在最后给一个风险分数，

比如 0.72。但是这个数字本身说不清楚患者为什么高风险，也说不清楚风险是突然升高，还是一直很高。

动态表型的做法是把这条曲线拆成 5 个滚动窗口。每个窗口都经过编码器，然后分到 P0、P1、P2、P3 中的一个状态。这样，一个患者就会得到类似 P0 到 P1 到 P2 的状态序列。这个序列可以表达病情轨迹。

在我的结果里，P2 被保留为最高风险且相对稳定的状态。软件里医生看到的不只是一个红色风险标签，而是可以看到这个患者当前更接近哪种表型，过去几个窗口有没有恶化，以及历史库里有没有类似轨迹的患者。

这也是为什么我说表型是 communication layer。它把模型里的 embedding 翻译成更像人能理解的语言：低危稳定、炎症高反应、循环休克、呼吸衰竭。这个翻译不等于临床诊断，但它比单个分数更容易交流。

第 9 页：最终 S7 模型

这一页我想让老师记住：最终 S7 模型小、可打包、可部署，并且输出能服务多个界面。

可以这样讲：这一页再回到最终模型结构。

S7 的主体是 missingness-aware GRU。GRU 适合处理时间序列，缺失感知输入让它更适合 ICU 数据。训练目标里有 source-balanced GroupDRO，用来避免某一个数据源完全主导训练；有 phenotype CE，让模型对主源表型保持一致；还有 supervised contrastive regularization，让同类表型的表示更靠近。

这个模型只有 31,319 个参数。这个数字不大，但对课程交付反而是优点。因为它更容易放进模型服务，更容易打包，更容易解释，也更适合做 smoke test。项目里真正复杂的地方不在参数量，而在数据接口、表型读出、转移矩阵、API 服务和交付验证。

我在这里想说的不是“我的模型最大”，而是“这个模型刚好能支撑系统需要的输出”。医生端需要风险和轨迹，研究端需要表型和相似病例，训练终端需要模型状态和 artifact，部署脚本需要可验证结果。S7 是为了这些使用场景选择出来的。

第 10 页：关键结果与证据边界

这一页我想让老师记住：结果要讲，但边界更要讲；主动说清楚，比答辩时被追问更好。

可以这样讲：这一页是结果汇总。

最终 S7 在课程演示监控切分上，encoder 加 transition 的 Macro F1 是 0.956，full-patient match rate 是 0.832，死亡 AUROC 是 0.848，下一时段机械通气 AUROC 是 0.968，剩余 ICU 住院时长 MAE 是 2.955 小时。

这些数字可以这样理解：表型 Macro F1 和 full-patient match rate 说明模型能较好复现动态表型轨迹；死亡 AUROC 说明它能提供一定的风险排序能力；机械通气 AUROC 和 LOS MAE 说明它不是只学了一个标签，而是还保留了一些辅助临床任务能力。

和早期方法相比，规则 baseline 只有 0.142，S6 v2 是 0.821 左右，GroupDRO 是 0.806 左右，而 S7 encoder 到 0.971，S7 smooth 到 0.956。这个提升说明最终加入表型监督和监督对比后，动态表型一致性明显更好。

但这一页最重要的一句话其实是限制：这些指标支持课程演示和工程 smoke test，不支持我声称它已经通过独立临床验证。因为 S7 monitoring split 与训练患者有重叠，所以我不能把它说成 held-out external validation。我的表述会是：它是一个 demo-ready、evidence-backed 的课程模型，但不是临床投产模型。

过渡句：研究部分讲完以后，下面进入软件系统。也就是说，模型不仅要有结果，还要被真正包装成可用的软件。

3 软件系统：从模型到可用产品

第 11 页：多角色产品体验

这一页我想让老师记住：SepsisCare 不只是模型接口，而是围绕医生、研究者、家属、管理员设计了不同 workflow。

可以这样讲：这一页是产品体验。因为 SepsisCare 最终不是给模型研究者单独看的，而是要展示一个医疗 AI 系统在不同角色下怎么工作。

医生端主要解决“现在病人怎么样”。它展示当前患者队列、风险等级、48 小时趋势、表型状态和解释信息。医生不应该看到一堆训练日志，而是应该看到和当前患者有关的风险线索。

研究端解决“这个结果有没有历史依据”。它提供历史 ICU 病例库、相似病例检索、细节面板、数据集摘要和模型实验输出。这样老师在演示时可以看到，系统不是凭空输出，而是能回到历史轨迹和表型结果。

家属端解决“怎么把复杂病情说得更容易懂”。这里接入了 DeepSeek 兼容的智能体，但它的定位是非诊断性沟通，不替代医生，不下医嘱，也不编造没有的数据。

管理员端和训练终端解决“系统有没有在线、模型能不能维护”。它们负责服务状态、模型配置、训练动作、日志、artifact 下载和继续训练。

所以这页我想强调：同一个模型服务，面对不同角色要给不同粒度的信息。这个项目的工程价值就在这里。

第 12 页：软件架构与 API 流

这一页我想让老师记住：多端能工作，靠的是统一 HTTP API 和可替换模型服务，而不是每个平台各写一套逻辑。

可以这样讲：系统架构这页可以用一句话概括：客户端轻一点，后端稳一点，模型服务可替换。

macOS、Windows、Android 和 Web 客户端都通过标准 HTTP API 访问后端。客户端不直接读模型文件，不直接碰 DeepSeek，也不直接管理训练状态。所有请求先到 `server.py`，再由后端分发到患者列表、历史库、模型预测、家属聊天、训练终端和管理员状态。

这样设计有几个实际好处。第一，多端行为更一致。比如 Windows、Android、iOS/Web 的共享前端资源可以同步更新，不需要每个平台单独维护一套业务逻辑。第二，模型可以替换。课堂演示时可以跑本地 packaged backend；远端部署时，训练终端可以把动作转发到云端 model service。第三，安全边界更清楚。客户端拿不到模型文件，也不直接保存敏感配置。

所以我说，可部署系统需要稳定 API contract。模型权重当然重要，但如果没有稳定接口，多端软件很快会变成几套互相不一致的 demo。

第 13 页：网络安全加固

这一页我想让老师记住：我把“本地演示方便”和“远端敏感接口安全”分开处理，避免把演示密码误当生产安全。

可以这样讲：安全部分第一块是网络安全。

一开始做课堂 demo 时，本地访问要方便，最好双击就能跑。但一旦系统有远端模型服务、训练命令、artifact 下载和审计日志，就不能再用“本地 demo 方便”的逻辑处理远端接口。

所以最终版本里，远端敏感 API 是 fail-closed 的。也就是说，远端客户端访问训练命令、artifact、审计、历史导出等敏感路径时，必须配置强 Bearer token。如果没有 token，或者 token 是弱口令、占位符、太短，服务会拒绝。缺失或错误授权头会返回 401，未配置或弱 token 会返回 403。

前端也做了对应收紧。API token 不再持久化到 `localStorage`，只保存在会话里；如果启动 URL 里带了 token 参数，加载后会清理掉。DeepSeek 和训练终端的 URL 也做了 SSRF 防护，拒绝 metadata service、link-local、带凭据的 URL 等危险目标。敏感响应还会设置 no-store 和 nosniff。

这里我会特别说明：**123123** 只是客户端演示入口，不是生产 API 鉴权。换句话说，就算你知道演示密码，也不能直接调用远端训练或 artifact 接口。这个边界很重要，因为很多 demo 项目最容易把演示便利性误写成安全机制。

第 14 页：数据污染与 Artifact 完整性

这一页我想让老师记住：交付包要能证明自己干净：没有明显敏感数据、没有训练元数据误导、artifact 能校验。

可以这样讲：安全部分第二块是数据污染和 artifact 完整性。

我把检查分成几层。第一层是 runtime data audit。它检查演示数据里有没有 provenance，是否混入 subject id、hadm id、姓名、MRN、DOB、地址等直接标识；还会检查重复 ID、孤立 detail series 和临床数值范围。

第二层是 sensitive surface audit。它扫描模型包、后端、Web、Windows、iOS、Android 六个发布面，找高置信密钥、Bearer token、PHI-like 文本等。为了避免二次泄漏，审计结果只输出文件、行号和短 hash，不回显原始敏感内容。

第三层是 training metadata audit。它检查四个数据来源、声明训练人数、目标 run、split policy 和指标范围。这个检查很有必要，因为如果训练元数据写错，很容易把 monitoring split 误宣传成 held-out validation。

第四层是 artifact verifier。模型服务提供的 ZIP artifact 不是随便打包一个目录，而是从已知模型和报告根目录重新构建，排除 runtime logs、配置文件和 symlink，并写入 MANIFEST.sha256，可选 HMAC。下载后可以逐文件验证。

当前结果是：runtime fatal violations 为 0，sensitive-data findings 为 0，training metadata violations 为 0，声明训练患者数 347,634。我要强调，这说明交付包做了 release hygiene，但不等于正式 HIPAA expert determination，也不等于原始数据治理已经完成。

第 15 页：部署编排

这一页我想让老师记住：部署不再是一串手动命令，而是一条可以测试、可以拉回 artifact、可以说明失败边界的流程。

可以这样讲：部署这页讲的是工程闭环。

早期 demo 只要本机能打开就可以。但如果要做 final delivery，就需要把部署步骤固定下来。现在的流程是：先跑 pre-release gate，确认安全检查、数据审计、前后端测试和构建检查通过；然后做 SSH preflight；再 rsync 同步模型服务包；远端执行 one-click doctor/start/smoke；Mac 侧做 API forwarding smoke；最后拉回 artifact 并验证。

这个流程里还有一些运维细节。比如 dry-run 日志会把 Bearer token redacted 掉，避免复制命令时泄漏 token。break-glass 跳过 pre-release gate 必须写原因，并写入 owner-only 审计文件。artifact 拉回后可以用 manifest 和 HMAC 验证。

当前 ready state 是：本地 app、API、model service 和部署编排脚本是可复现、可测试的；ROG production-port 路径也有验证证据。但我不会把所有远端环境都说成已经完全生产上线，因为真正独立 remote server 还需要更多跨机器端到端证据。这个边界写在部署报告里，也会在汇报里主动说明。

第 16 页：最终交付物

这一页我想让老师记住：最终交付不是一个文件，而是软件、模型包、文档和保证脚本四类东西。

可以这样讲：最终交付物可以分成四类。

第一类是软件。包括 macOS app、Windows installer、Android/Web 资源、启动脚本和本地 demo 相关文件。用户拿到以后，可以按 quick start 启动本地 API 和模型服务。

第二类是模型部署包。里面有 S7 模型权重、表型读出器、转移矩阵、模型服务代码、runtime

database 和 one-click 脚本。这个包的作用是让模型服务可以从本机迁移到远端服务器，而不是绑死在我的电脑上。

第三类是文档。包括论文报告、最终 PPT、安装指南、部署说明、transfer README、SHA256 manifest 和边界说明。文档不是装饰，它是验收时用来追踪“这个东西从哪里来、怎么安装、怎么验证”的。

第四类是保证脚本。包括 pre-release gate、runtime data audit、sensitive surface audit、training metadata audit、artifact verifier、smoke test 和服务编排脚本。

为什么要拆这么细？因为这样可以把原始数据和 API key 排除在安装包之外，可以让模型服务独立部署，也可以让老师在验收时看到明确的证据链。最终交付的目标不是“我现场能演示”，而是“别人拿到交付包也能理解和验证”。

4 收束：主动讲限制，反而更可信

第 17 页：限制与下一步

这一页我想让老师记住：最后不要只说项目很完整，要诚实说清楚它离临床使用还差什么。

可以这样讲：最后这一页我会用三个 gap 收束。

第一个是 validation gap。当前 S7 monitoring split 适合课程交付和 smoke test，但它不是独立 held-out clinical validation，因为它和训练患者有重叠。下一步如果要走向更严肃的临床研究，就必须构建真正外部验证切分，或者拿独立医院数据做严格验证。

第二个是 deployment gap。现在本地服务、ROG 路径、部署脚本和 artifact 验证都有证据，但真正独立 remote server 还需要完整跨机器 proof。比如从另一台机器访问 API、运行 smoke logs、拉回 artifact、保存 verifier output。这些证据要补齐以后，才能说远端部署完全闭环。

第三个是 governance gap。现在项目已经做了 token gate、SSRF 防护、数据最小化、审计、manifest 和安全门禁，但如果要进入真实医疗环境，还需要 formal threat model、密钥轮换、IRB 或伦理审批、临床安全验证、数据保留策略和更完整的运维监控。

所以我最后的总结会这样说：SepsisCare 展示的是一条 course-grade 的完整路径，从 ICU 时间序列建模，到动态表型解释，再到多平台软件交付和安全证据。它可以演示，可以复现，也有明确边界。但它还不是临床设备。对我来说，这个项目最重要的收获就是：模型结果、软件体验和交付安全必须一起做，任何一个缺了，系统都不完整。

谢谢各位老师。

5 现场压缩与临场提醒

如果只有 8 到 10 分钟

1. 第 1 页：只讲“研究模型 + 工作软件 + 安全证据”，用 30 秒。
2. 第 2 页：讲清楚课程演示边界，不展开医疗器械。
3. 第 3 页：只讲四个数据源和 347,634 总数。
4. 第 4 到第 7 页：合成一段，“先表示、再表型、再鲁棒训练”。
5. 第 8 到第 10 页：重点讲动态表型和结果边界。
6. 第 11 到第 12 页：用 1 分钟讲多角色产品和 API 架构。
7. 第 13 到第 15 页：合成一段，“token、审计、artifact、部署脚本”。
8. 第 16 页：20 秒说明交付包结构。
9. 第 17 页：必须保留，主动讲限制。

最容易讲错的地方

- 不要说“已经临床验证”。应该说“课程演示监控切分和工程 smoke test 支持交付验证”。
- 不要说“123123 是安全密码”。应该说“它只是客户端演示门，远端敏感 API 由 Bearer token 控制”。
- 不要说“打包了全部原始 ICU 数据”。应该说“交付包包含脱敏演示 runtime 数据、模型产物和报告，不包含受限原始数据库”。
- 不要把 ROG、本地服务、独立 remote server 混成一个概念。可以说“本地和 ROG 路径已有验证，独立 remote server 仍需跨机器证据”。

可能被问到的问题

问题	建议回答
为什么不直接做死亡预测？	因为项目目标不是只给一个死亡概率，而是服务一个多角色系统。动态表型可以解释患者状态怎么变，也能支撑相似病例检索和医生端趋势展示。
S7 的指标能不能说明临床可用？	不能直接说明。S7 monitoring split 用于课程交付、模型比较和 smoke test，和训练患者有重叠。临床可用需要独立 held-out 或外部医院验证。
为什么强调缺失感知？	ICU 数据的缺失经常和采样频率、病情严重程度、医院流程有关，不是普通表格里的随机空值。所以 mask 和时间差本身可能提供临床信号。
GroupDRO 在这里解决什么？	它让模型在多来源训练时不要只优化平均损失，也关注表现较差的数据源。这样比简单把所有数据混在一起更稳。
安全审计做到什么程度？	做到工程 release hygiene，包括 token gate、SSRF 防护、session token、HTML escaping、runtime audit、sensitive surface audit、training metadata audit 和 artifact manifest。它不等于 HIPAA、IRB、FDA 或医疗器械认证。
最终交付最重要的价值是什么？	不是某一个模型分数，而是把模型、API、客户端、训练终端、部署脚本和安全证据接成了可复现链路，并且把边界写清楚了。